

Lab 5: Logic Equivalence Checking (LEC) using Cadence Conformal

R Sarang

October 5, 2026

Contents

1	Introduction	2
2	Task 1: Baseline Verification	2
3	Task 2: Bitwise Operator Typo	2
4	Task 3: Multiplexer Signal Swap	3
5	Task 4: Missing Enable Logic on Flip-Flops	4
6	Task 5: Hardcoded Reset Pin	6
7	Task 6: Incorrect Primary Output Wiring	8
8	Task 7: Bad Clock Gating	10
9	Task 8: Renamed Internal Wires (Unmapped Points)	12
10	Task 9: Corrupted Combinational Logic	12

1 Introduction

This report documents the verification of an ALU Accumulator Core using Cadence Conformal Logic Equivalence Checking (LEC). The design was verified across 9 distinct tasks. For each task (except the baseline Task 1), a bug was injected into the `revised_netlist.v` file. The tool was used to identify non-equivalent points (NON-EQ), trace the error to its root cause using the Diagnosis Manager and Schematic Viewer, and correct the revised Verilog code to match the Golden design.

2 Task 1: Baseline Verification

Objective: Establish a baseline by comparing an unmodified revised netlist with the golden netlist.

Diagnosis & Correction: The comparison yielded 0 NON-EQ points. The designs were 100% structurally and logically equivalent. No bugs were present.

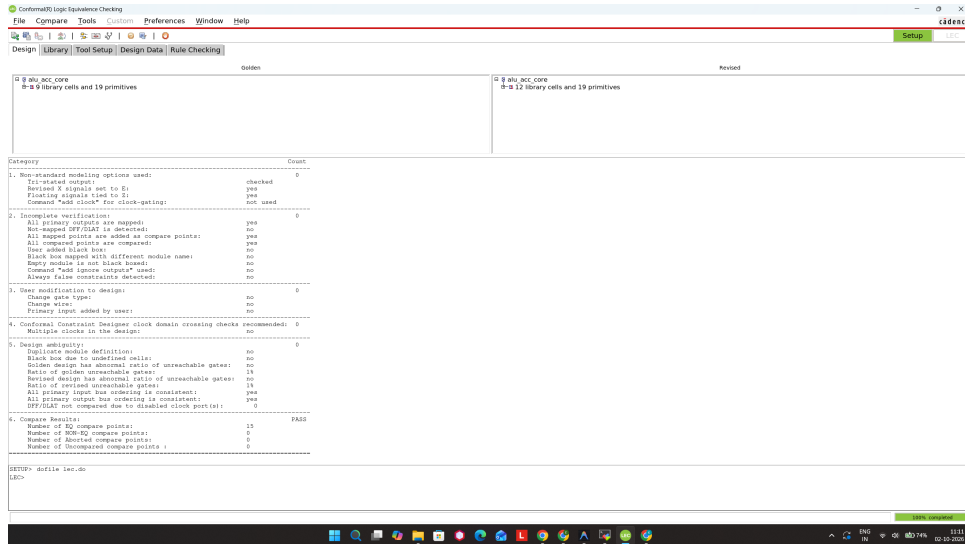


Figure 1: Task 1 - 100% Equivalent (PASS)

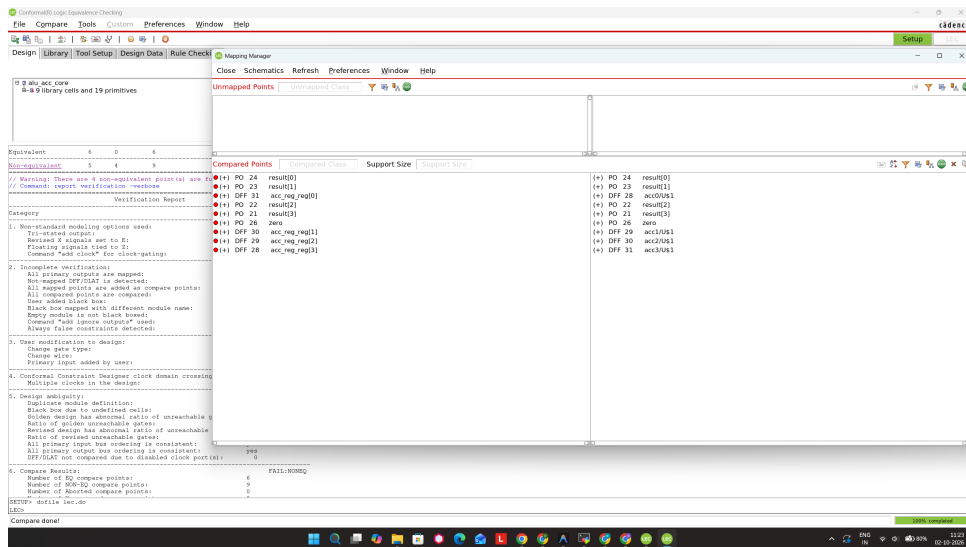
3 Task 2: Bitwise Operator Typo

Objective: Identify and correct a logical bug in the ALU combinational logic.

Diagnosis: Conformal identified failing primary outputs. The designer accidentally used a bitwise OR operator (`|`) instead of a bitwise AND operator (`&`) for the `and_r` signal.

Correction: The operator was corrected in `revised_netlist.v`.

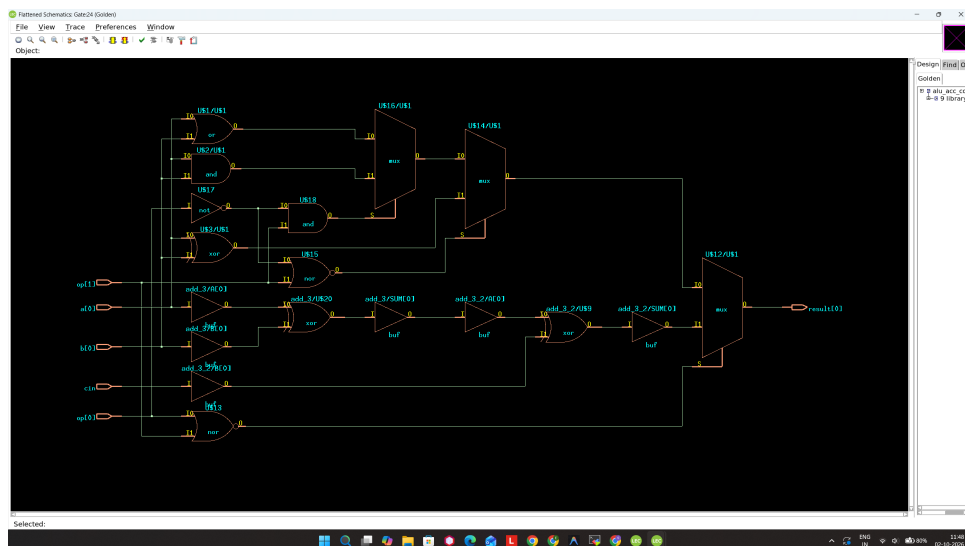
```
1 // Original Broken Code:
2 assign and_r=a|b;
3
4 // Corrected Code:
5 assign and_r=a&b;
```



4 Task 3: Multiplexer Signal Swap

Objective: Debug a corrupted output caused by misrouted signals in a multiplexer.

Diagnosis: The LEC tool flagged cascading errors across `result`, `zero`, and `acc_reg`. Diagnosing the primary output `result` revealed that the inputs to the output multiplexer for opcodes 10 and 11 were swapped.



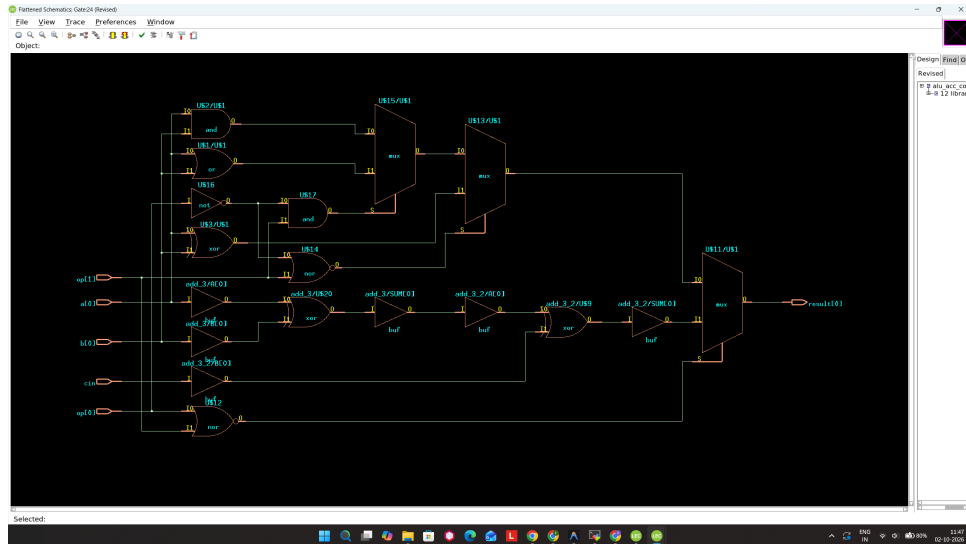


Figure 4: Task 3 - Revised Schematic showing swapped wires

Correction:

```
1 // Original Broken Code:
2 assign result=(op==2'b00)?add_r:(op==2'b01)?xor_r:(op==2'b10)?or_r:
  and_r;
3
4 // Corrected Code:
5 assign result=(op==2'b00)?add_r:(op==2'b01)?xor_r:(op==2'b10)?and_r:
  or_r;
```

Category	Count	Status
1. Non-standard modeling options used:	0	checked
2. Incomplete verification:	0	yes
3. User modification to design:	0	no
4. Conformal: Constraint Designer clock domain crossing checks recommended:	0	no
5. Design integrity:	0	no
6. Compare Results:	15	PASS

Figure 5: Task 3 - PASS after correction

5 Task 4: Missing Enable Logic on Flip-Flops

Objective: Identify missing sequential logic controlling the flip-flops.

Diagnosis: The primary outputs passed, but all 4 `acc_reg` flip-flops failed. The Golden design used a multiplexer to loop the output back into the flip-flop when the `en`

signal was 0. The Revised design ignored this completely.

Golden		Revised	
0 3 alu_acc_core 8-9 library cells and 19 primitives		0 3 alu_acc_core 8-8 12 library cells and 15 primitives	
Equivalent	11 0 11		
Non-equivalent	0 4 4		
// Message: There are 4 non-equivalent point(s) are functional mapped. // Command: report verification -reasons			
Verification Report		Count	
1. Non-standard modeling options used:		checked	0
Truncated output	no	yes	
Revised 9 signals set to 0	yes	yes	
Floating signals tied to 0	yes	yes	
Command: warn clock for clock-gating	no	yes	
2. Incomplete verification:			0
All primary outputs are mapped	yes		
Not-mapped OFF/SLAT is detected	no		
All mapped points are added as compare points	yes		
All compared points are compared	yes		
User added black box	no		
Black box mapped with different module name	no		
Design is not black boxed	no		
Constant and control signals of nodes	no		
Always false constraints detected	no		
3. User modification to design:			0
Change gate types	no		
Primary input added by user	no		
4. Conformal Constraint Designer clock domain crossing checks recommended:			0
Multiple clocks in the design	no		
5. Design ambiguity:			0
Duplicate module definition	no		
Black box due to undefined cells	no		
Golden design has abnormal ratio of unreachable gates	no		
Ratio of golden unreachable gates	15		
Revised design has abnormal ratio of unreachable gates	24		
Ratio of revised unreachable gates	24		
All primary inputs has consistent is consistent	yes		
All primary outputs has consistent is consistent	yes		
OFF/SLAT not compared due to disabled clock port(s)	yes		
6. Compare Results:			Fail:NONEQ
Number of EQ compare points	11		
Number of NON-EQ compare points	4		
Number of Aborted compare points	0		
Number of Uncompared compare points	0		
RETURN deffile lec.do END			

Figure 6: Task 4 - Failing Flip-Flops in Mapping Manager

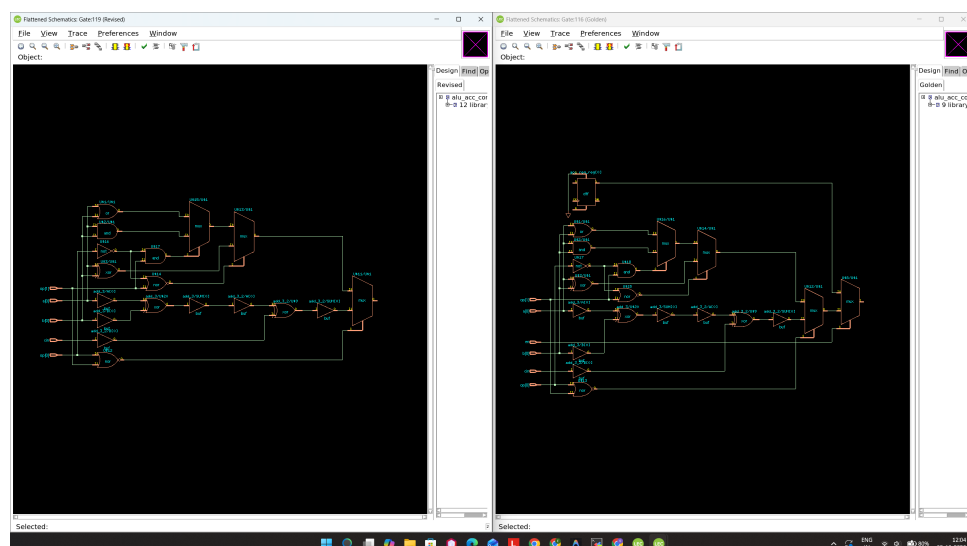


Figure 7: Task 4 - Schematic showing missing enable MUX in Revised

Correction: A ternary operator was added to the .D pin of all 4 flip-flops.

```
1 // Corrected Code:
2 DFFRX1 acc0(.D(en?result[0]:acc_reg[0]),.RN(rst_n),.CK(clk),.Q(acc_reg[0]));
```

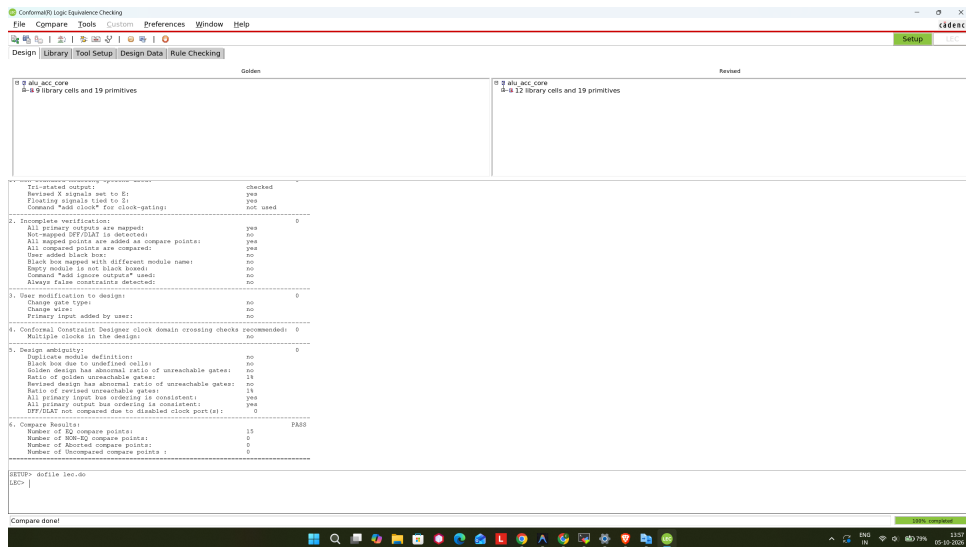


Figure 8: Task 4 - PASS after correction

6 Task 5: Hardcoded Reset Pin

Objective: Debug an improperly constrained reset pin on a sequential element.

Diagnosis: Only a single flip-flop (`acc_reg[2]`) failed. The schematic viewer showed that the RN (Active-Low Reset) pin was tied to a constant logic '1' instead of `rst_n`.

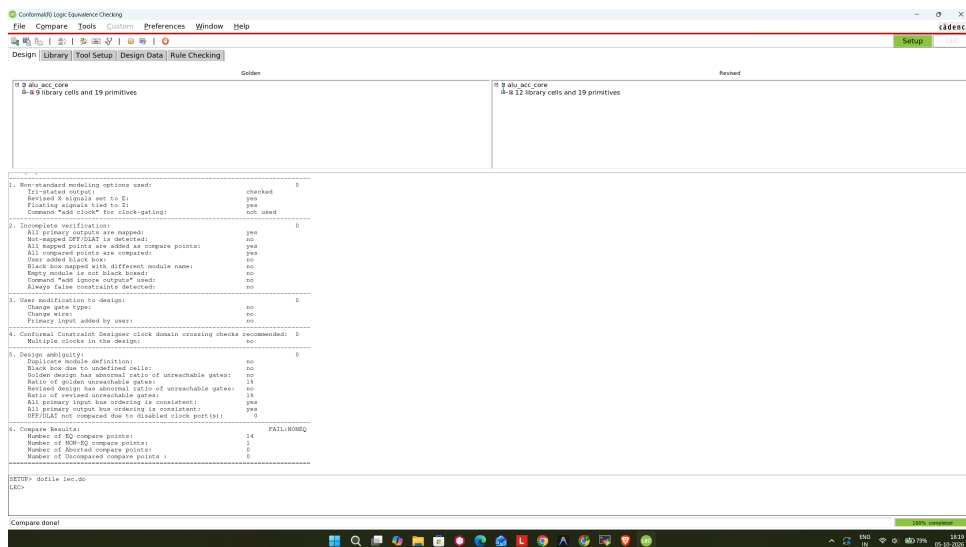


Figure 9: Task 5 - Initial Failure

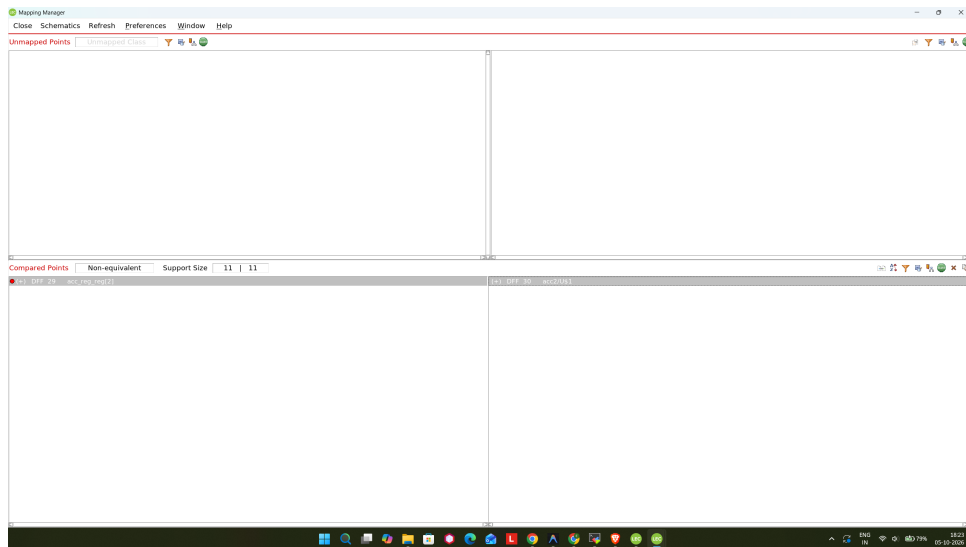


Figure 10: Task 5 - Mapping Manager isolating the single DFF

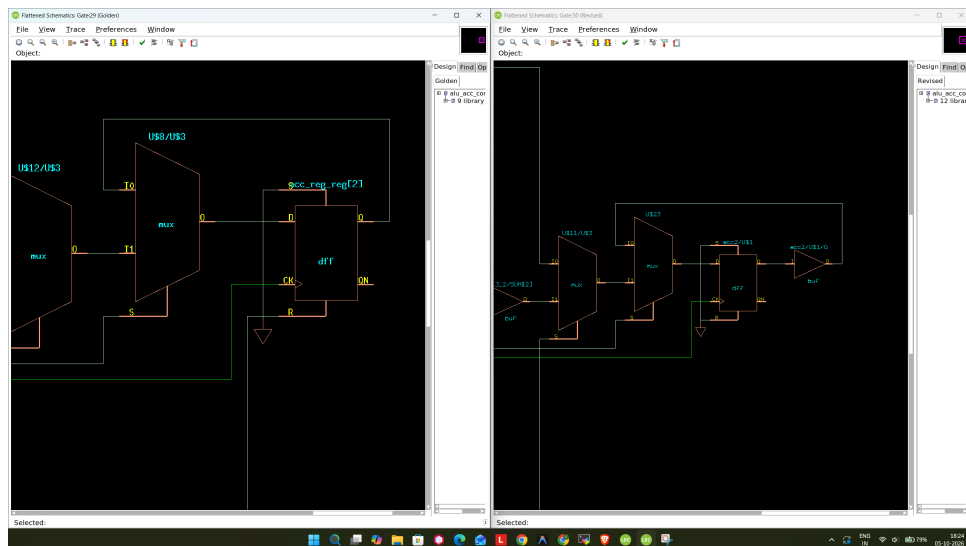


Figure 11: Task 5 - Schematic showing hardcoded reset pin (Logic 1)

Correction: The pin connection was restored to the `rst_n` net.

```
1 // Original Broken Code:
2 DFFRX1 acc2(.D(en?result[2]:acc_reg[2]),.RN(1'b1),.CK(clk),.Q(acc_reg
  [2]));
3
4 // Corrected Code:
5 DFFRX1 acc2(.D(en?result[2]:acc_reg[2]),.RN(rst_n),.CK(clk),.Q(acc_reg
  [2]));
```

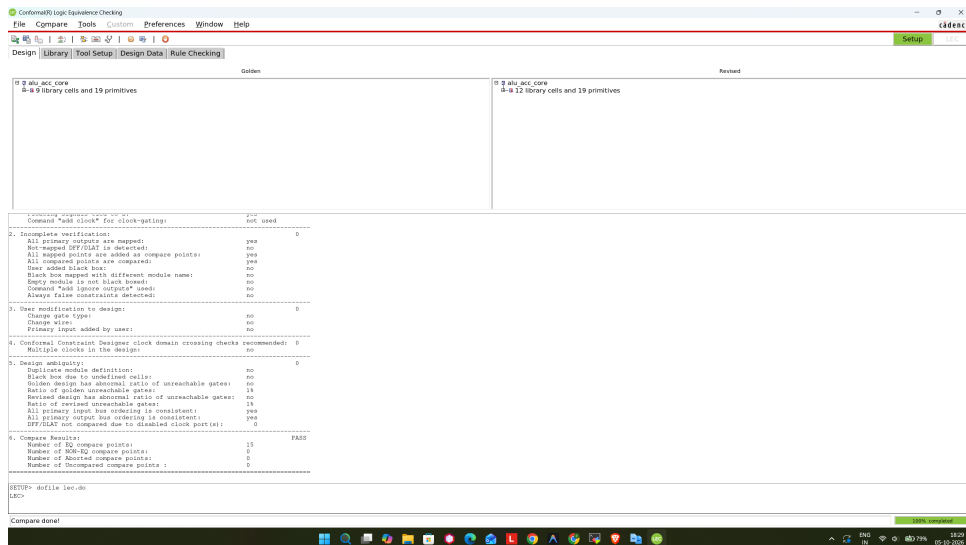


Figure 12: Task 5 - PASS after correction

7 Task 6: Incorrect Primary Output Wiring

Objective: Fix a primary output that was wired to the wrong input source.

Diagnosis: The `test_status` primary output failed. In the Golden design, this pin is hardwired to logic 0. In the Revised design, it was accidentally connected to the `test_mode` input pin.

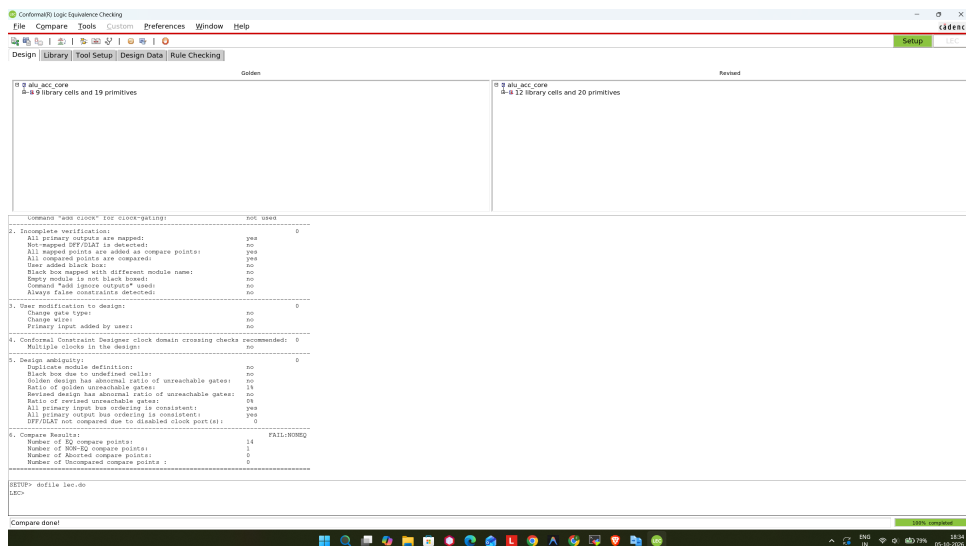


Figure 13: Task 6 - Initial Failure

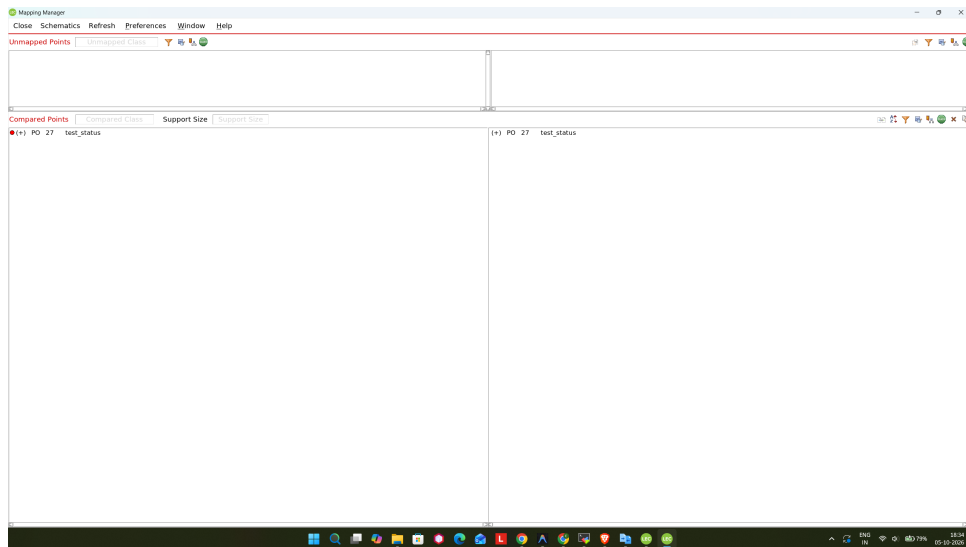


Figure 14: Task 6 - Mapping Manager pointing to test_status

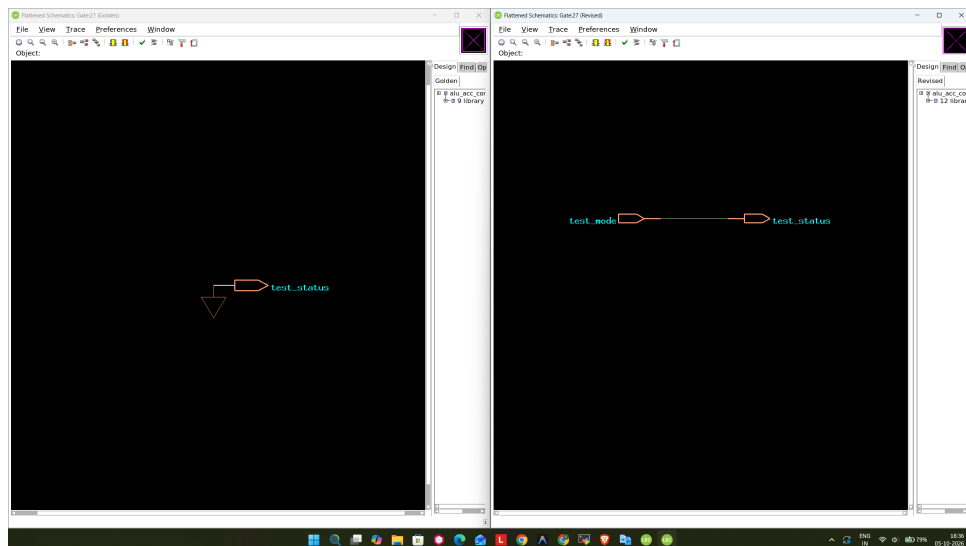


Figure 15: Task 6 - Schematic showing bad wiring

Correction:

```
1 // Corrected Code:
2 assign test_status=1'b0;
```

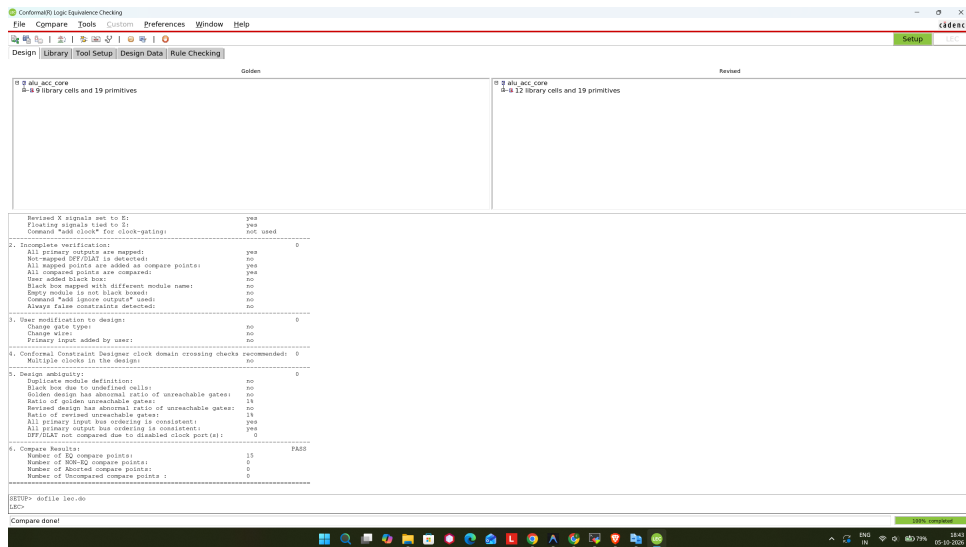


Figure 16: Task 6 - PASS after correction

8 Task 7: Bad Clock Gating

Objective: Identify equivalence failures caused by manually implemented clock gating.

Diagnosis: All 4 flip-flops failed. The designer attempted to implement clock gating using an AND gate (assign gated_clk = clk & en).

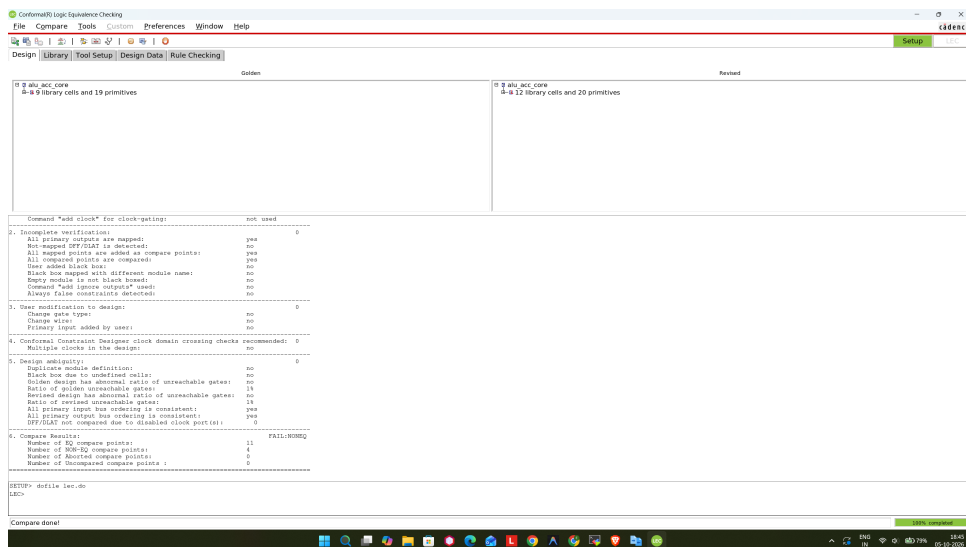


Figure 17: Task 7 - Initial Failure

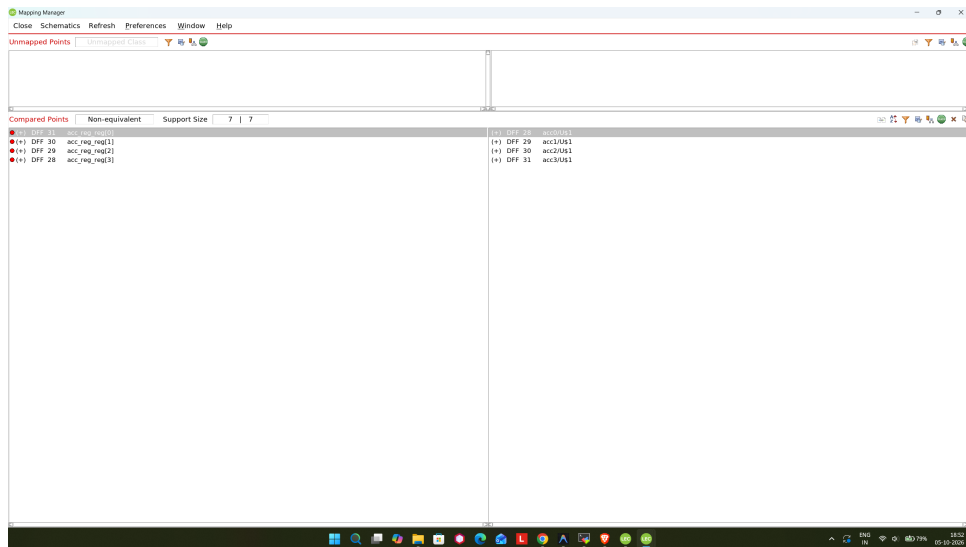


Figure 18: Task 7 - Mapping Manager showing all DFFs failed

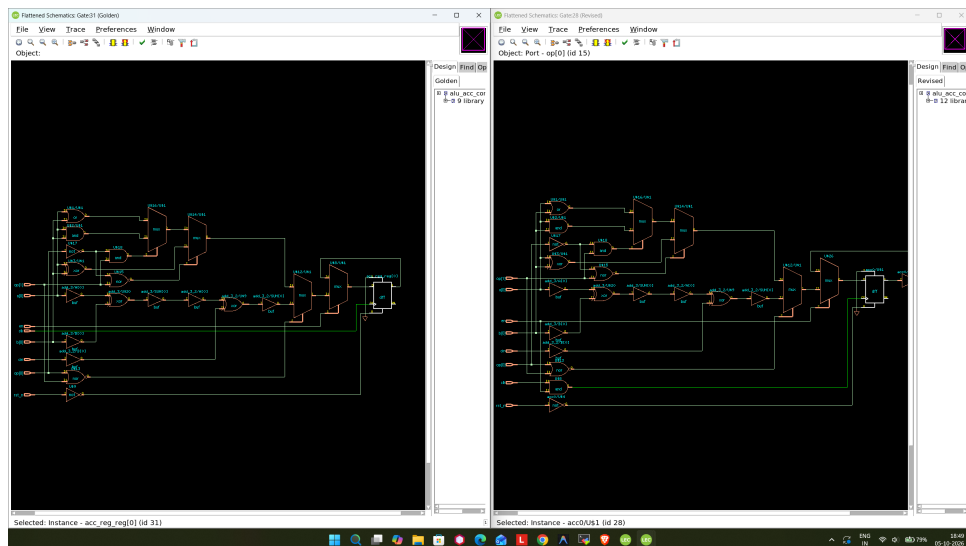


Figure 19: Task 7 - Schematic showing bad clock gating AND gate

Correction: The clock gating was bypassed to match the golden design.

```
1 // Corrected Code:
2 assign gated_clk=clk;
```

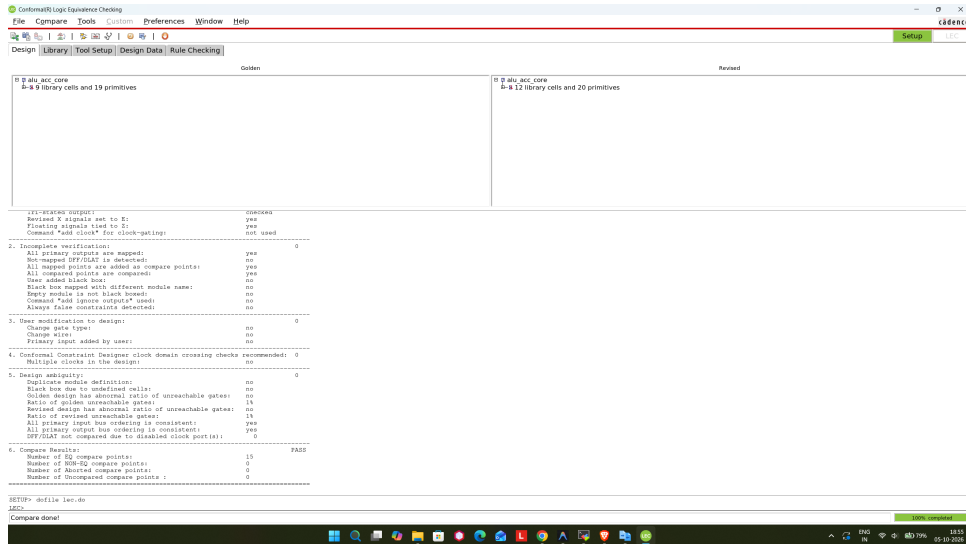


Figure 20: Task 7 - PASS after correction

9 Task 8: Renamed Internal Wires (Unmapped Points)

Objective: Understand structural mapping capabilities in Conformal.

Diagnosis: The designer renamed `acc_reg` to `accumulator_reg`. Conformal's structural mapping algorithms successfully mapped the logic cones and passed the design because it is logically identical.

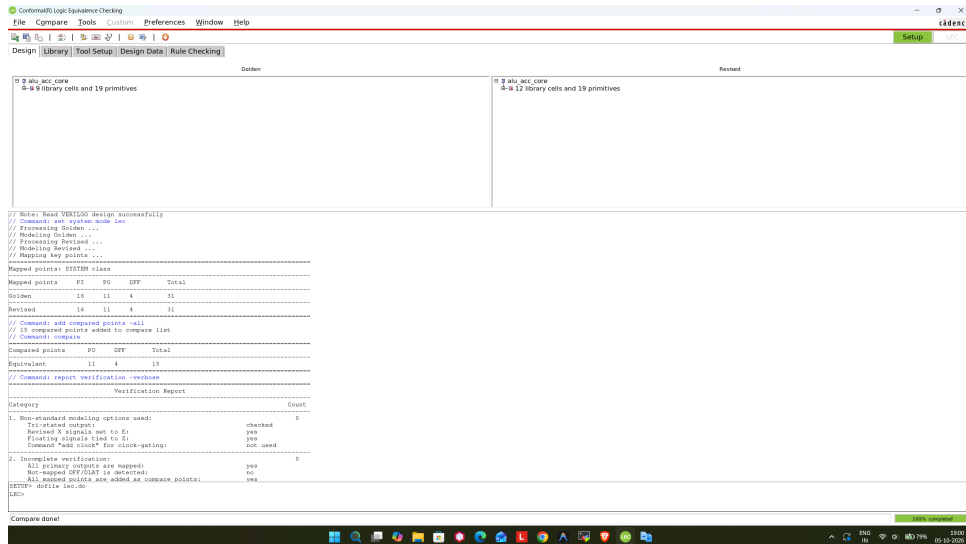


Figure 21: Task 8 - PASS with Structural Mapping despite renamed wires

10 Task 9: Corrupted Combinational Logic

Objective: Debug a hardcoded bit failure deep within the combinational logic cone.

Diagnosis: A cascading failure occurred on `result[2]`, `zero`, and `acc_reg[2]`. The designer hardcoded bit 2 of the AND operation.

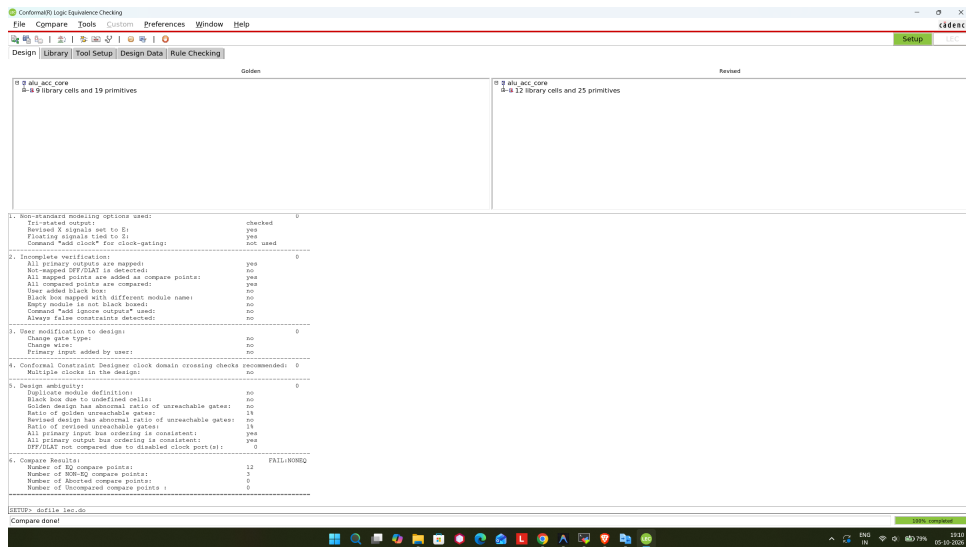


Figure 22: Task 9 - Initial Failure showing 3 cascading errors

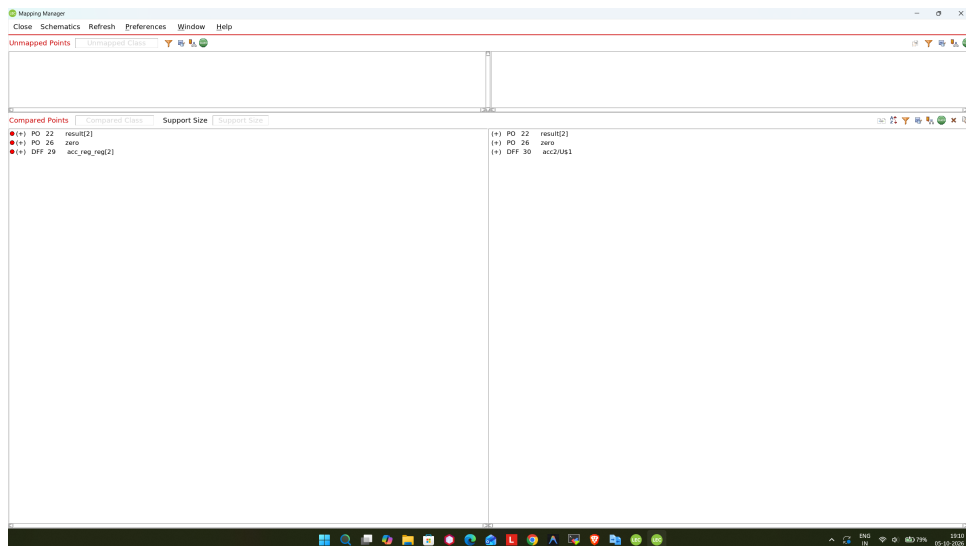


Figure 23: Task 9 - Mapping Manager

Correction: The redundant logic was deleted.

```
1 // Corrected Code:
2 assign and_r=a&b; assign or_r=a|b;
```

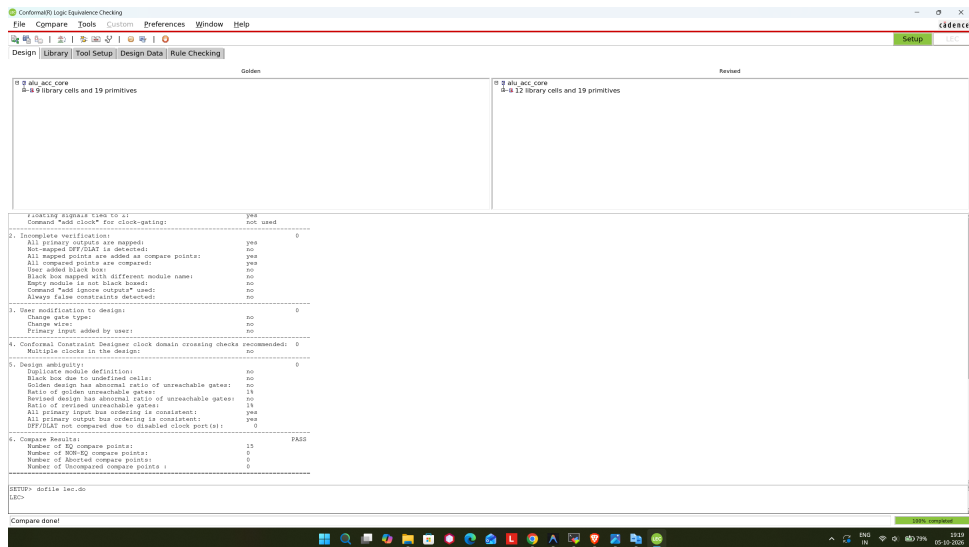


Figure 24: Task 9 - PASS after correction